

SoK: Analysis of Software Supply Chain Security by Establishing Secure Design Properties

Chinenye Okafor*

Purdue University
West Lafayette, IN, USA
okafor1@purdue.edu

Santiago Torres-Arias

Purdue University
West Lafayette, IN, USA
santiagotorres@purdue.edu

Taylor R. Schorlemmer*

Purdue University
West Lafayette, IN, USA
tschorle@purdue.edu

James C. Davis

Purdue University
West Lafayette, IN, USA
davisjam@purdue.edu

ABSTRACT

This paper systematizes knowledge about secure software supply chain patterns. It identifies four stages of a software supply chain attack and proposes three security properties crucial for a secured supply chain: transparency, validity, and separation. The paper describes current security approaches and maps them to the proposed security properties, including research ideas and case studies of supply chains in practice. It discusses the strengths and weaknesses of current approaches relative to known attacks and details the various security frameworks put out to ensure the security of the software supply chain. Finally, the paper highlights potential gaps in actor and operation-centered supply chain security techniques.

CCS CONCEPTS

• **Software and its engineering** → *Design patterns*; • **Security and privacy** → *Software security engineering*; • **General and reference** → *Surveys and overviews*.

KEYWORDS

Software Supply Chain Attacks, Security Properties, Collaborative Software Engineering, Software Reuse

ACM Reference Format:

Chinenye Okafor, Taylor R. Schorlemmer, Santiago Torres-Arias, and James C. Davis. 2022. SoK: Analysis of Software Supply Chain Security by Establishing Secure Design Properties. In *Proceedings of the 2022 ACM Workshop on Software Supply Chain Offensive Research and Ecosystem Defenses (SCORED '22)*, November 11, 2022, Los Angeles, CA, USA. ACM, New York, NY, USA, 10 pages. <https://doi.org/10.1145/3560835.3564556>

1 INTRODUCTION

Industrial, government, and academic computing systems rely on a supply chain of open- and closed-source software components [60]. An actor controlling any step in this chain may, accidentally or maliciously, sabotage downstream software [59, 63]. Problems in

software supply chains have caused site- and Internet-wide disruptions at an estimated cost of billions of dollars [12, 57]. These problems include service outages [15, 20] and cybersecurity exploits that endanger human lives [25] and national security [24]. Software supply chain exploits may be attributed to requirements mismatch — many software supply chains were originally designed for sharing, not cybersecurity [60]. In light of the emerging requirement for security, how should supply chains be designed?

Researchers have begun by structuring knowledge of how current software supply chains can be attacked. For example, taxonomies of attacks yield attack trees that help us understand how attackers compromise supply chains. Ecosystem analysis has also helped understand how the structure of software dependencies can make us more or less vulnerable when selecting external dependencies [19, 80, 81]. Data-science based efforts in the industry and academia have attempted to identify signals or indicators of compromise [79].

Other researchers have examined design changes to improve the security of software supply chains. Using these insights, efforts have focused in developing systems and mechanisms to mitigate these attack vectors. Efforts like in-toto and Sigstore attempt to provide a layer of security to the current operations in the software supply chain. These generally aim to protect against a class of attacks in the software supply chain. For example, Solarwind's Trebuchet project [24] aims to prevent compiled backdoors by means of an in-toto coordinated reproducible builds-based pipeline. This has led to the development of "meta-frameworks" or "best practices models" that describe a combination of mechanisms and configurations that can be used to provide a strong security posture against software supply chain attacks. Examples of these approaches are the Cloud Native Computing Foundations Technical Advisory Group on Security's (CNCF TAG-Security) reference architecture for secure software pipelines [31], as well as the Secure Software Factory [32].

Given the emerging nature of this discipline and the amount of disjoint efforts, various groups from academia, industry and open source have proposed multiple reference architectures and design patterns to secure their software supply chain. However, as yet there is no a systematized framework that helps system integrators and designers to understand and match how these mechanisms and different architectures are designed and applied in popular software pipelines. This is in part due to the disconnect between these different communities, but as well as a lack of structured

*Both authors contributed equally to this research.



This work is licensed under a Creative Commons Attribution International 4.0 License.

SCORED '22, November 11, 2022, Los Angeles, CA, USA

© 2022 Copyright held by the owner/author(s).

ACM ISBN 978-1-4503-9885-5/22/11.

<https://doi.org/10.1145/3560835.3564556>

framework to map both research aims: to catalogue threats and to apply a combination of systems to mitigate them.

The purpose of this paper is to summarize knowledge regarding design patterns for security in software supply chains. We systematically review current design patterns for secure supply chains, and develop a framework to compare their security postures. In doing so, we provide the first comprehensive study of current best practices proposed by industry, academia and government. First, we describe software supply chains (§2). Next, we provide a four stage attack pattern for software supply chain attacks (§3.1). Then, we present three properties (transparency, validity, and separation) for securing software supply chains (§3.2). Afterwards, we document how current security practices fulfill our security principles (§4) and provide several case studies (§5). Finally, we identify opportunities to further improve the security of software supply chains (§6).

In summary, our contributions are:

- (1) An attack pattern for software supply chain attacks (§3.1).
- (2) Principles for a secure software supply chain (§3.2).
- (3) A collection and analysis of current security practices (§4 and §5).

2 BACKGROUND

In general, a supply chain is a set of entities which interact to produce some product for an end consumer [67]. Each link in a supply chain contributes to the final product by providing a sub-product to reliant links. As a result, a network of dependencies forms between the links in a supply chain - terminating with the link representing the end consumer. Therefore, a supply chain is characterized by the connections and attributes of the entities used to create and ultimately consume a final product.

In computing, **software supply chains are a collection of systems, devices, and people which produce a final software product** [18]. Figure 1 depicts a typical software supply chain with a focus on an individual link. Each *link* in the software supply chain comprises the artifacts, operations, and actors needed to develop and deliver software products [21, 52]. *Actors* manipulate artifacts and operations within the supply chain to produce an output. *Artifacts* include the product team's code, development infrastructure, and software dependencies. *Operations* include productive steps such as fetching dependencies or compiling software, protective steps such as linting or security scans, and publishing steps such as deployment or distribution.

The structure of supply chains necessitates an interdependence between artifacts and operations within and between links. Actors manage the connections which form between components and between links in the chain. Responsibility for operations and artifacts is distributed among actors across different geographies, teams, companies, and legal jurisdictions. Modern software engineering is an international collaborative effort [34, 64]. A single link in the supply chain does not necessarily correspond to one group or organization. A single organization may be responsible for several links within a supply chain.

3 SUPPLY CHAIN ATTACKS AND SECURITY PROPERTIES

3.1 Supply Chain Attacks

The software supply chain is an increasingly popular attack vector [43]. It is comprised of several connected links which share artifacts and conduct operations. Actors manage links and components. The difference between software supply chain attacks and other software attacks, however, is not clearly defined in literature. Ladisa *et al.* [43] and Ohm *et al.* [57] characterize supply chain attacks as the injection of malicious code into the supply chain to target downstream links. ENISA [18] defines a supply chain attack as a combination of at least two attacks — one attack on a supplier and a subsequent attack on intended targets. Other works such as Zimmermann *et al.* [81] and Zahan *et al.* [79] identify methods other than strict code injection for supply chain attacks. **Distilling the concept of software supply chain attacks from multiple sources, we arrive at a characteristic four stage attack pattern shown in Figure 2:**

- (1) **Compromise:** First, an attacker finds and compromises an existing weakness within a supply chain.
- (2) **Alteration:** Second, an attacker leverages the initial compromise to alter the software supply chain.
- (3) **Propagation:** Third, the change introduced by the attacker propagates to downstream components and links.
- (4) **Exploitation:** Finally, the attacker exploits the alterations in a downstream link.

To illustrate this definition, consider the SOLARBURST compromise [11, 49]. In this supply chain attack, an attacker altered existing software from SolarWinds by injecting malicious code during the build process. This attack can be mapped to the four-stage attack from Figure 2 as follows: (1) The *existing weakness* compromised was the build infrastructure. (2) The *alteration* was malicious code injected by the compiler, permitting a user to bypass authentication in a SolarWinds product component. (3) The *propagation* was via SolarWinds's compromised product — its users include many companies and US government agencies like the IRS and NASA. (4) The *exploitation* was to leverage broken authentication mechanism to take control of affected machines. Such incidents are becoming common; software supply chain compromises have increased by a cumulative 650% in the last three years [66, 68, 69].

In contrast to this pattern, traditional attacks, such as those described by Lockheed Martin's Cyber Kill Chain [46], simply exploit an existing vulnerability (step 4). Attacks on software are not necessarily supply chain attacks just because the software exists within the context of a supply chain. For this reason, *an attack on software via the weakness of a dependency is not a supply chain attack unless it follows the attack pattern*; the attacker must both introduce the upstream change *and* subsequently exploit it downstream.

For another perspective on this attack pattern, consider the distinction between vulnerable and malicious dependencies as drawn by the European Union Agency for Cybersecurity (ENISA) [18] and Ohm *et al.* [57]. Vulnerable links in the supply chain contain unintended weaknesses that may be exploited further downstream. These exploits are not supply chain attacks. On the other hand, malicious links in the supply chain were intentionally designed to

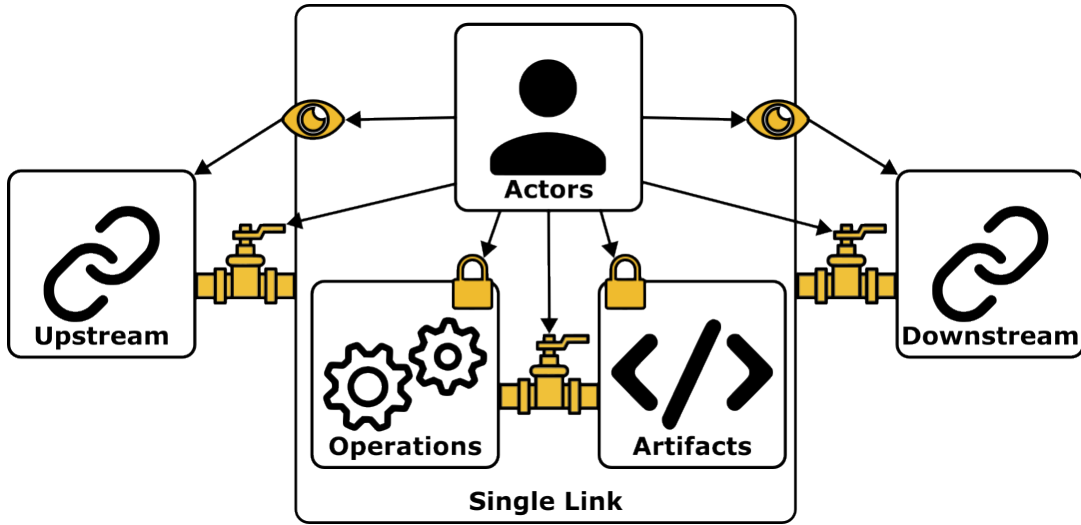


Figure 1: A software supply chain with focus on a single link. Actors manage components and connections within and between links. Therefore, actors manage security. Security depends on upstream and downstream transparency, link validity via component integrity and actor authentication, and logical separation between components and links.

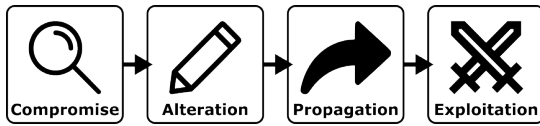


Figure 2: Four stage software supply chain attack pattern. Attackers begin with an initial *compromise* before making some malicious *alteration* to the supply chain. This change then *propagates* down the supply chain where attackers *exploit* the introduced weakness(es).

weaken the rest of the chain. Introducing and subsequently exploiting such weaknesses constitutes a supply chain attack.

Existing literature categorizes and documents known supply chain attacks [18, 43, 57, 81]. It is outside of the scope of this paper to enumerate individual attack types. Typically, this line of work differentiates between how attackers compromise and alter the supply chain.

3.2 Security Properties for Software Supply Chains

Components of a supply chain must be secured to mitigate the presence of vulnerabilities and the risk of attack. Supply chains become secure when attackers are unable to compromise components, alter the supply chain, or propagate malicious changes. **In the literature on software supply chain security, we have identified three orthogonal and recurring security properties:**

- (1) **Transparency:** Although actors only control portions of a supply chain, increased knowledge of the entire chain allows all parties to mitigate risk or employ specific countermeasures against an attack [13, 22, 23, 43, 48]. Transparency represents the availability of that knowledge to actors in the supply chain. Transparency applies to the entities connecting and comprising links in the chain.

- (2) **Validity:** Software supply chains should remain correct. Changes to actors, operations, or artifacts in a single link can compromise downstream entities. Validity comprises integrity of operations, integrity of artifacts, and authentication of actors. Each link in the supply chain contains a series of operations and artifacts which interact with other links in the chain. Secure supply chains require that these components remain unchanged by malicious parties [13, 23, 48]. Therefore, only authorized actors should make changes to link connections and components. Such changes must also receive permission to occur [23, 72, 79].
- (3) **Separation:** Secure supply chains embody a compartmentalized nature. Connections are an integral part of supply chains, but should only exist when necessary. These connections should be minimized to reduce attack surface area. Additionally, logically separate operations, artifacts, and actors should remain separate in practice to minimize unintended connections. By implementing measures such as mirroring, version locking, containers etc. individual components can decrease reliance on security of others [43, 48, 79].

3.3 Analysis of Security Properties

The security properties discussed in §3.2 are only meaningful if, when applied ideally, they eliminate the risk of attack. For this to be the case, properties must be comprehensive.

To analyze these properties, we consider a hypothetical attack following the pattern discussed in §3.1 and apply transparency, validity, and separation. Since defenders can typically only address the first three stages of attack, we show how applying these properties prevents an attack from reaching the final stage: exploitation. Lastly, we note the difference between ideal conceptualizations and real-world embodiments of these security properties.

First, transparency primarily protects against the first stage of attack. Transparency, in its ideal state, enables perfect vision of all

actors, operations, and artifacts across the supply chain. Such transparency would allow managers of a supply chain to identify link weaknesses before they are compromised. By securing weaknesses through patches, fixes, or other methods, managers block attempts at *compromise*. By identifying weaknesses first, managers prevent attackers from completing the first stage.

Second, validity primarily protects against the next stage of attack: *alteration*. By maintaining perfect integrity of operations, integrity of artifacts, and authentication of actors, no unauthorized changes can be made to the supply chain. As a result, attackers have no ability to maliciously alter the supply chain.

Finally, separation primarily protects against the third stage of attack: *propagation*. If a supply chain system can perfectly compartmentalize and moderate interactions between entities, then malicious changes cannot propagate downstream. In this case, connections between artifacts, operations, and actors are managed in such a way that malicious changes cannot affect other supply chain components. With ideal separation, only valid changes (e.g., system updates and patches) can traverse the supply chain.

By preventing at least one of the three stages leading to exploitation, a supply chain attack cannot occur. While this hypothetical considers an ideal case, practical application is not so easy. Real techniques typically do not fully realize security properties, but they provide partial coverage (e.g., attestations do not necessarily provide complete transparency). Real techniques also do not always implement security properties independently. In practice, techniques might require cohesion between multiple security properties. For example, a technique might require transparency to identify threats propagating through the supply chain before applying separation methods. Conversely, achieving close-to-ideal transparency may only be possible if the supply chain is sufficiently separated from other entities.

Since techniques do not perfectly embody security properties, defending in depth is critical [40]. Theoretically, a single technique could prevent all attacks if it provided a perfect implementation of transparency, validity, or separation. In practice, techniques have flaws. For this reason, using multiple techniques mapped to each of the security properties provides a more effective defense against attack.

4 MAPPING PROPOSALS TO SECURITY PROPERTIES

In this section, we map proposals to secure the software supply chain against the security properties presented in §3.2.

4.1 Promoting Transparency

A lot of work has been done on creating transparency in the software supply chain. To mitigate the security risks associated, it is crucial to have information about the various software components and dependency as well as their hierarchical relationship in the supply chain [2, 50]. A primary tool for transparency is the Software Bill of Materials (SBOM) [6]. A SBOM is an inventory of all the components that make up a software product. SBOM promotes transparency by tracking component metadata, enabling mapping to other sources of information, and tying the metadata to software as it moves down the supply chain and is deployed [70]. As

security vulnerabilities are discovered in these components, applications that depend on them can be quickly and reliably tracked and updated to newer, hardened versions.

An SBOM provides a foundation for additional capabilities that enhance software supply chain security, e.g., Component Analysis. Component Analysis is a function within an overall Cyber Supply Chain Risk Management (C-SCRM) framework as it helps to understand and manage risks that these components may present to the missions they support. As the software supply chain extends due to the increasing system complexities, it is increasingly important to understand, evaluate, and manage the risk that various components in the supply chain may present in addition to its function. For example, the criticality of artifacts and processes could be used to determine data dependency between components. Building on a set of multidisciplinary publications, standards, and guidelines [14, 36–39, 41, 62], Paulsen *et al.* [58] proposed a Criticality Analysis Process Model that prioritizes programs, systems, and components based on their importance to the goals of an organization and the impact that their inadequate operation or loss may present to those goals.

Some techniques provide SBOM-similar information to users. For example, Sigstore's [51] transparency log gives users the ability to view information about artifacts and operations used to create them; npm-audit [55] gives users a way to visualise dependencies; and git commit signing [27] provides authorship information. Several other tools automate information collection and assurance across the supply chain.

The collection of this information builds trust within the supply chain. By understanding how artifacts, operations, and actors interact, members of the supply chain can trace dependencies back to a trusted root source. This enables the flow of trust between entities in the supply chain. The promotion of validity ensures that the interactions between supply chain elements are certifiable — increasing trust.

4.2 Promoting Validity

Establishing trust entails providing security at every step, which incrementally evolves, each built on the preceding to provide incremental confidence. While end-users know what components make up their software, can they trust what the component says it is? Furthermore, believe the integrity of the executable received is intact? High reliance on open and closed source packages has decreased the confidence that systems only do what they are intended for. Manual code review helps to identify vulnerabilities, but since some software are distributed as prebuilt binaries, it is less effective to manually review the individual source code for malicious flaws.

Lamb *et al.* [44] and Goswami *et al.* [30] looked at the problem of establishing trust in build artifacts by comparing build outputs from multiple independent builders [42, 47]. This way, the user can verify that the received binaries are identical to other builds. The reproducible build approach is a countermeasure solution to attacks that could compromise the executables at build time, where changes are essentially invisible to its original authors and users alike. Code signing and verification is an integral part of ensuring that software is from an original publisher. It ensures that the final published software is intact and contains no tampering from unauthorized parties. Sigstore [51] improves the integrity of the software supply

chain by combining various technologies to provide an automated approach for developers to digitally sign artifacts and for users to verify the artifacts in Sigstore’s transparency log - a public, tamper-proof ledger of signatures. This mapping of artifacts to verifiable identities establishes trust that they are tamper-free. Gitsign implements keyless Sigstore to sign git commits with a valid OpenID Connect identity which overcomes the challenges associated with using GPG keys in signing git commits and consequently improving the overall trust in open-source projects. Many point solutions have been implemented [3, 7, 27, 75] to ensure that individual supply chain actions are not altered. Torres-Arias *et al.* [73] implemented a holistic approach that enforces the integrity of a software supply chain by gathering cryptographically verifiable information about the chain itself and verify that each step action of a supply chain is not tampered with. This approach ensures end-to-end verification and confirms that tampering does not occur in between steps in the software supply chain between the development and the publication of the software.

Account takeover attacks place the account owner and anything the account has access to at risk. Multi-factor Authentication is highly recommended for actors in the supply chain. Although all downstream systems that depend on the affected code are impacted, no solution provides the security posture of actors (e.g. maintainer, developer) in a supply chain to enable software consumers to make risk-based security decisions. Will it have more cons than pros? Several C-SCRM practices for systems and organizations have been recommended to mitigate attacks due to credential compromise. Building on existing FIPS 200 standards [56], Boyens *et al.* [13] added that information system access should be limited to only the necessary type and duration and monitored for cybersecurity supply chain impact. They also expanded the Awareness and Training control of FIPS 200 to include providing C-SCRM awareness and training to individuals at all levels within the enterprise as well as suppliers, developers, system integrators, external system service providers, and other information technology (IT)- or operational technology (OT)-related service providers to ensure that the personnel who interact with an enterprise’s supply chains receive the training as appropriate.

4.3 Promoting Separation

Several techniques currently exist for promoting separation between components in a supply chain. We distinguish between those techniques that apply to internal components and those that apply to external links. Internal-focused techniques ensure compartmentalization between artifacts, operations, and actors of a single link. External-focused techniques mitigate the risk associated with relying on other links in the supply chain.

First, container and virtual machine (VM) based methods separate internal operations, artifacts, and actors [23, 48, 65]. Successful build systems are highly automated [31]. In these systems, automated workers can behave as actors by managing operations and artifacts. In these cases, best practice is the creation of ephemeral and task-specific workers [31]. This prevents any unnecessary crossover between logically different operations and actors - reducing the risk associated with internal connections. Additionally, the creation of compartmentalized containers and VM instances reduces the risk

associated with connected artifacts. As an example, BreakApp [74] assists in spawning compartmentalized modules to complete tasks during development. By systematically creating separated build systems, attackers cannot effectively propagate vulnerabilities throughout the supply chain.

Second, actors adopt systems which provide separation from external supply chain links. For example, version locking and mirroring techniques add a layer of security to external sources [21, 31, 48]. Version locking ensures that a link includes a particular version of an upstream component. The use of a constant dependency version ensures that malicious changes upstream do not automatically propagate to downstream links. A weakness to version locking is the reliance on actors to accurately set and manage version numbers. The failure to update versions may prevent updates that remove vulnerabilities. In the same manner, prematurely updating to a compromised version defeats the purpose of version locking. Mirroring acts in a similar manner to version locking. Organizations create private package feeds to mitigate the risk of pulling dependencies from public sources [48, 76]. This gives organizations more control over the import of packages into their software projects. Once again, this relies on including the correct packages in private feeds.

5 MAPPING EMBODIMENTS TO SECURITY PROPERTIES

In this section, we use case studies to demonstrate how each security property from §3.2 can be embodied in practice. We specifically consider efforts to secure supply chains through (1) package repositories, (2) development environments, (3) end-to-end solutions, and (4) security frameworks.

5.1 Package Repositories

Package and dependency managers such as npm for JavaScript, PyPI for Python, or RubyGems for Ruby have encouraged code reuse between packages. Consequentially package managers have become a vital part of software supply chains. Attackers have begun exploiting the weak links in these ecosystems to distribute malware. Several techniques have been put in place to detect and mitigate package manager ecosystem attacks. We examine Npm as a case study to discuss the techniques that have been adopted to improve the security properties defined in §3.2.

Npm is the package manager for the popular Node.js JavaScript platform. Since Node.js is widely used for back-end software, Npm is widely used in security-sensitive contexts.

Transparency: To use Npm as a package manager, a software engineer must declare the packages on which they depend in a file called *package.json*. The command *npm install* command can then install these packages. The *package.json* file is a “primitive SBOM” since it contains almost all the required fields of one (as recommended by the NTIA [70]). SBOM’s in general, provide an example of the Transparency property in practice.

As one application of this primitive SBOM, Npm offers a tool called *npm-audit* [55] to audit package security and stability. Npm-audit assesses the dependency description of a project against the default registry for possible security vulnerabilities and calculates the impact and appropriate remediation if any are found. The audit process entails scanning the *package.json* and *package-lock.json*

file to build the dependency tree, then comparing the packages from the dependency tree to a database of known vulnerabilities. If any vulnerabilities are found, an alert with the impact and appropriate remediation will be shown. Although security audits help you protect package users by enabling you to find and fix known vulnerabilities in dependencies that could cause data loss, service outages, and unauthorized access to sensitive information, recent researchers have started doubting the reliability of this tool [45, 78].

Validity: Npm has recorded several incidents on the registry where npm accounts are hijacked by malicious actors and the use of the access to infiltrate packages the compromised accounts have access to. Following the unprecedented series of account takeovers resulting from the compromise of developer accounts without 2FA enabled [5, 16], npm has introduced even more security enhancements. An extra layer of security of verifying all npm account login, and enrolling maintainers of top packages into a mandatory 2FA have been added to help prevent common accounts takeover attacks, such as credential stuffing [71], which utilize a user's compromised and reused password.

Separation: Npm provides scopes to safeguard a diverse set of package names by restricting the package's namespace to an organization or user [54]. This means that one does not have to worry about someone taking a package name ahead of time and only the user is allowed to publish packages under that scope on the public registry, which hardens the process against compromise as an attacker would have to compromise that npmjs.org registry account to take over the package. Scopes can be associated with a given registry which ensures that all requests for packages under the scope will be routed to the given registry. For instance, at login the *myorg* scope can be linked to <http://registry.myorg.com> with the command npm login:

```
$ npm login --scope=@myorg
--registry=http://registry.myorg.com
```

This command will ensure that any request bound to the *myorg* scope is sent to the <http://registry.myorg.com> registry. Scopes mitigate the dependency confusion risks where an Internal package name is claimed by an attacker on the public registry.

The use of npm proxy [53] is a common practice for improved npm security. The internal registry can be configured to take precedence over the public registry to help protect against installing the wrong or malicious package from public registries. For example, an attacker might publish a malicious package to the public repository with the same name as a package hosted on a private registry but with a higher semantic version. In the case where a custom setting for an internal registry is omitted, the package manager would default to the public registry and download the latest (malicious) packages from there. Configuring the proxy to never allow an upstream request to the public registries protects against fetching arbitrary packages in place of the legitimate package.

5.2 Development Environments

Collaborative development environments like GitHub and Bitbucket host millions of open- and closed-source projects. The meritocratic premise of open-source software [60], which allows contributions from anyone, is an opportunity for the introduction of vulnerabilities through “hypocrite commits” [77]. Hence, such platforms have

taken steps to promote the security of the projects they host. We align these steps with our proposed security properties in a case study of GitHub.

Transparency: In a package context such as Npm, the package manager can mandate the use of a *package.json* file. This file can be leveraged for further analysis. In contrast, in the source code context, the supply chain for a given project is harder to discover. GitHub [4] identified unpatched software as the major threat to supply chain security and has provided capabilities of Software Composition Analysis (SCA) to determine dependencies in use, discover vulnerabilities in the dependencies, and effect patches. These capabilities are provided by dependency graph, dependabot alerts, and dependabot security updates feature and are recommended to organizations to help secure their repositories against supply chain threats [29].

The dependency graph is a summary of manifests and lock files that shows the dependencies and dependents of your repository. When a pull request containing changes to dependencies that target the default branch is created, GitHub uses the dependency graph to add dependency reviews to the pull request. These indicate whether the dependencies contain vulnerabilities and, if so, the version of the dependency in which the vulnerability was fixed. Dependabot alerts rely on the dependency graph and GitHub advisory database to alert developers when a repository is affected by a newly discovered vulnerability. This enables organizations and open-source projects to stay up to date on security vulnerabilities, and information. Dependabot security updates make it easier to mitigate this vulnerability within repositories by automatically raising pull requests to update a software dependency to the minimum version that resolves a known vulnerability. These dependabot features provide automation to the hard work of dependency management and patching. However, the extent to which the dependency update bot reduces update suspicion and notification fatigue remain questionable [10, 33].

To eliminate the security risk posed by the late detection of vulnerabilities, GitHub [1] developed a feature — static analysis, powered by CodeQL, that runs queries against codebases to identify potential security vulnerabilities.

Validity: To secure build systems against build process attacks, Github Actions (CI/CD tool for GitHub) is designed to ensure precise and repeatable build steps and that each build starts in a new environment to reduce the likelihood of attackers persisting in a build environment [28]. GitHub builds on top of the git version control system, which has a feature to enable developers to validate that commits are coming from an identified, trusted source while using other people's work. Specifically, git supports signing and verifying commits and tags using GPG [27].

Separation: Platforms such as GitHub allow software developers to create arbitrarily many independent repositories (within reason). Links between these repositories (e.g., by adding a dependency or a git sub-module) are at the developer's discretion. This decision improves the separation between links in the supply chain. However, we note that the common practice of *vendoring* — copy-pasting a dependency verbatim into another codebase — degrades separation.

5.3 End-to-end Solutions

Some researchers have focused on developing fully-fledged systems to mitigate supply chain attacks. Solutions such as in-toto [73] and Sigstore [51] are excellent examples and have been integrated across vendors to secure software supply chains [9]. We examine in-toto as a case study for the implementation of security properties proposed in §3.2.

In-toto ensures the security of the software supply chain by gathering cryptographically verifiable evidence — called *link metadata* — about entities in the chain. *Link metadata* is a signed statement each actor in the supply chain emits to describe relevant operations, artifacts, and connections. For example, this statement may include information such as files used, files produced, building processes, or even environment variables. *Link metadata* is collected throughout the supply chain and is delivered alongside the final product. Verifiers can compare *link metadata* with a *layout* (created by an actor who dictates policy for the supply chain) describing the intended steps in the supply chain.

Transparency: By disaggregating the supply chain into small, individual claims from each actor, in-toto provides strong transparency guarantees for artifacts (they are tracked as materials and products) and operations (they are described in a layout file). However, it does not provide transparency for actors in the supply chain. Although actors will create signed statements, in-toto does not have a method to tie their keys to identities (even pseudonyms). While this is not exclusive to the “holistic” approach of the system, it may be possible to extend the design to include known identifiers using approaches such as verifiable credentials [17] or distributed identity providers [61].

Validity: Regarding validity, in-toto provides guarantees for artifact integrity (by means of hashing each artifact) and operation integrity (by means of a layout policy), yet it does not provide actor authentication. Even though the signing keys prevent a malicious takeover of operations carried out by an actor, it does not protect against other attack vectors (e.g., account take over). It is possible that author validity guarantees may be achieved by combining existing mechanisms (e.g., 2FA) into the signing flow for link metadata. Likewise, actors provide additional proof they are a well-known, reputable actor in the chain through techniques like DiD and VC described above.

Separation: Lastly, in-toto provides strong Separation guarantees for operations and actors, but not for artifacts. This is because artifact separation is typically achieved by underlying mechanisms (e.g., a container runtime may sandbox a build process). Given this, it may be possible to provide stronger security guarantees by adopting hardened runtimes within solutions like in-toto. One example of this is SLSA’s extension to in-toto links [26], that can help communicate information if the build is hermetic, or if all the artifacts used in the build were required by the build.

5.4 Security Frameworks

Researchers and industry have proposed multiple end-to-end security frameworks to simplify the process of securing a supply chain. These frameworks recommend practices, tooling options and design considerations to ensure the integrity of artifacts in the software

supply chain. We consider three frameworks — Microsoft’s Supply Chain Integrity Model (SCIM), Google’s Supply-chain Levels for Software Artifacts (SLSA) and The Cloud Native Computing Foundation’s (CNCF) Software Supply Chain Best Practices.

First, SCIM [8] specifies how the artifact verification process should function across a supply chain. The framework provides a standard for the artifact data model and exchange format (SCIM-Evidence), the policy used in evaluating these artifacts (SCIM-Policy), and the service that stores both the evidence and the policy (SCIM-Store). This enables the smooth flow of evidence (e.g., bills of materials, build information, etc.) between links in the supply chain.

Second, SLSA [26] provides four levels of assurance (i.e., SLSA 1-4) to describe the security posture of a supply chain, similar to the capability maturity model (CMM) for software development [35]. The framework starts with basic security at SLSA 1 and adds requirements at each level until the final level, SLSA 4. Each level specifically documents source, build, provenance, and common requirements.

Finally, the CNCF [31] framework provides a five-stage methodology for securing supply chains. These stages include securing the source code, materials, build pipelines, artifacts, and deployments of a supply chain.

In Table 1, we compare how the three frameworks cover our security properties. SCIM provides substantially less coverage than SLSA 4 and CNCF. Although SLSA 4 and CNCF cover our security properties better, they may be difficult to implement¹.

Table 1: Comparison of how SCIM [8], SLSA [26], and CNCF [31] promote security properties.

Frameworks		SCIM	SLSA 4	CNCF
Transparency	Artifacts	✓	✓	✓
	Operations	✓	✓	✓
	Actors		✓	✓
Validity	Artifacts	✓	✓	✓
	Operations	✓	✓	✓
	Actors		✓	✓
Separation	Artifacts		✓	✓
	Operations		✓	✓
	Actors			✓

Transparency: SCIM improves the supply chain’s transparency by providing principle for conveying evidence about the artifact. For example, information about the sub-components of an artifact, how the artifact was created, and defects identified in the artifact. SLSA promotes transparency by demanding provenance attestation files at the lowest compliance level. These files contain build metadata which inform users about the artifacts they use. The CNCF framework proposes several practices to increase transparency in the supply chain, such as SBOMs and dependency analysis.

Validity: SCIM improves the validity of the supply chain by providing principles for conveying the evidence of an artifact that allows it to be verified. For example, providing the principles for

¹SLSA 4 lists 20 requirements; the CNCF proposes nearly 60 requirements.

Table 2: Proposed and practical techniques, related to security properties (columns) and aspects of the definition of a supply chain (sub-columns). Note the emphasis on *artifacts* and the under-emphasis on *operations* and *actors*. We believe this gap represents an opportunity for security research. We suggest that vetting *operations* may require applied cryptography and improved hardware root-of-trust. Meanwhile, addressing security flaws related to *actors* will require accounting for human and organizational factors, a longstanding challenge in cybersecurity.

Techniques	Transparency			Validity			Separation		
	Artifacts	Operations	Actors	Artifacts	Operations	Actors	Artifacts	Operations	Actors
SBOM	✓	✓							
npm-audit [55]	✓			✓					
Code scanning [1]	✓			✓					
Dependabot features [29]	✓			✓					
GitHub Actions [28]		✓		✓	✓			✓	
Git Commit Signing [27]			✓	✓					
Scope [54]				✓			✓		✓
Multi-Factor Authentication						✓			
In-toto [73]	✓	✓		✓	✓			✓	✓
Containerization							✓	✓	✓
Version Locking							✓		
Sigstore [51]	✓	✓	✓	✓	✓				
Mirroring and Proxies [53]	✓			✓			✓	✓	

conveying the cryptographic hash of a software allows the software to be verified by the consumer. *SLSA 2 to 4* generates validity requirements for implementers. At these levels, *SLSA* demands version control, provenance integrity, auditability, and a two-party review of all changes to the artifact. Such specifications ensure that artifacts and operations within the supply chain are tamper-free. The *CNCF* framework recommends a combination of cryptographic attestation and verification at each stage of the supply chain (verification by reproducibility, Multi-factor authentication, and signature validation at every step, amongst others) to ensure integrity.

Separation: The *SCIM* framework does not provide specific methods to increase separation in the supply chain. *SLSA 3* requires isolated builds in ephemeral environments (dedicated resources for that particular build) and *SLSA 4* requires a hermetic build process. These requirements provide strong protection against cross-build contamination attacks [26]. Also, the *CNCF* framework recommends several practices that embody separation, including maintaining controlled source code, build, and artifact environments. Controlled environments are created by adopting policies such as branch protection rules, ephemeral build workers, ephemeral certificates, pipeline orchestration, minimal network connectivity, build worker segregation, and artifact access rules.

6 DISCUSSION

In §4 and §5 we related current proposals and embodiments to the security properties from §3.2. We believe this demonstrates the usefulness of the properties we have proposed. As a last question, we consider whether these approaches are comprehensive with respect to software supply chains. Recall the definition of a software supply chain given in §2: *a collection of systems, devices, and people which produce a final software product* Figure 1.

We mapped each technique described in §4 and §5 to the corresponding security properties and the relevant aspect(s) of the software supply chain. Table 2 presents our results. We observe that

most of the studied approaches are focused on artifacts. Further research is needed to assess the generalizability of this claim beyond the cases we examined. If true, we acknowledge that artifacts are the appropriate primary focus for security — when deployed, software systems consist of interacting software artifacts, and cybersecurity vulnerabilities consist of exploiting these interactions. However, we suggest that there are many opportunities to improve the handling of operations and actors in secure software supply chains.

7 CONCLUSION

In this paper, we proposed desired properties of a secure software supply chain and systematized current design patterns and practices. We analyzed security frameworks and several real-world techniques and mapped these techniques according to the corresponding principles they promote. Our analysis showed how current practices embody the proposed security properties and apply to components of the software supply chain. We intend for this systematization to serve as a reference and guide for those seeking to build frameworks and improve the security of software supply chains.

ACKNOWLEDGMENTS

We acknowledge support from Cisco as well as NSF award #2229740. We thank the reviewers and our shepherd, Asra Ali, for their thoughtful critiques.

REFERENCES

- [1] About code scanning. <https://docs.github.com/en/code-security/code-scanning/automatically-scanning-your-code-for-vulnerabilities-and-errors/about-code-scanning>.
- [2] Cyclonedx is sbom: Software bill of materials. <https://cyclonedx.org>.
- [3] Reproducible builds. <https://reproducible-builds.org/>.
- [4] Secure at every step: What is software supply chain security and why does it matter? <https://github.blog/2020-09-02-secure-your-software-supply-chain-and-protect-against-supply-chain-threats-github-blog/>.
- [5] Security issue: compromised npm packages of ua-parser-js (0.7.29, 0.8.0, 1.0.0) - questions about deprecated npm package ua-parser-js · issue #536 · faisalman/ua-parser-js. <https://github.com/faisalman/ua-parser-js/issues/536>.
- [6] Software Bill of Materials. <https://www.cisa.gov/sbom>.
- [7] The Update Framework (TUF). <https://theupdateframework.github.io/>.
- [8] SCIM: Supply chain integrity model. <https://github.com/microsoft/scim>, 2022. Accessed: 2022-09-14.
- [9] In-toto ongoing integrations, Retrieved July 30. <https://in-toto.io/integrations/>.
- [10] Enable dependabot by milgradesec · pull request #4317 · caddyserver/caddy, Retrieved July 31. <https://github.com/caddyserver/caddy/pull/4317>.
- [11] Catalin Cimpanu · Microsoft, fireeye confirm solarwinds supply chain attack. <https://www.zdnet.com/article/microsoft-fireeye-confirm-solarwinds-supply-chain-attack/>.
- [12] A. Cherepanov. *Analysis of TeleBots' cunning backdoor*. <https://www.welivesecurity.com/2017/07/04/analysis-of-telebots-cunning-backdoor>.
- [13] J. Boyens, A. Smith, N. Bartol, K. Winkler, A. Holbrook, and M. Fallon. Cybersecurity Supply Chain Risk Management Practices for Systems and Organizations. Technical Report NIST Special Publication (SP) 800-161 Rev. 1, National Institute of Standards and Technology, May 2022.
- [14] J. M. Boyens, C. Paulsen, R. Moorthy, and N. Bartol. Supply chain risk management practices for federal information systems and organizations. <https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-161.pdf>.
- [15] Chris Williams. How one developer just broke node, babel and thousands of projects in 11 lines of javascript. https://www.theregister.com/2016/03/23/npm_left_pad_chaos/.
- [16] CISA. Malware discovered in popular NPM package, ua-parser-js. <https://www.cisa.gov/uscert/ncas/current-activity/2021/10/22/malware-discovered-popular-npm-package-ua-parser-js>.
- [17] W. W. Consortium et al. Verifiable credentials data model 1.0: Expressing verifiable information on the web. <https://www.w3.org/TR/vc-data-model/#core-data-model>, 2019.
- [18] E. U. A. f. Cybersecurity. ENISA threat landscape for supply chain attacks. Technical report, Publications Office, LU, July 2021.
- [19] A. Decan, T. Mens, and E. Constantinou. On the impact of security vulnerabilities in the npm package dependency network. In *International Conference on Mining Software Repositories (MSR)*, 2018.
- [20] A. Dellavechia. How a Rogue Developer Ruined Millions of Software (happened this weekend), Jan. 2022.
- [21] R. J. Ellison, J. B. Goodenough, C. B. Weinstock, and C. Woody. Evaluating and mitigating software supply chain security risks. 2010.
- [22] R. J. Ellison, J. B. Goodenough, C. B. Weinstock, and C. Woody. Evaluating and Mitigating Software Supply Chain Security Risks. Technical report, CARNEGIE-MELLON UNIV PITTSBURGH PA SOFTWARE ENGINEERING INST, May 2010. Section: Technical Reports.
- [23] M. Ensor and D. Stevens. Shifting left on security - Securing software supply chains. Technical report, Google Cloud, Feb. 2021.
- [24] FireEye. Highly evasive attacker leverages solarwinds supply chain to compromise multiple global victims with sunburst backdoor. <https://www.fireeye.com/blog/threat-research/2020/12/evasive-attacker-leverages-solarwinds-supply-chain-compromises-with-sunburst-backdoor.html>.
- [25] Forbes. Supply chains are in the crosshairs of cyberattacks. <https://www.forbes.com/sites/forbestechcouncil/2022/04/27/supply-chains-are-in-the-cyberattack-crosshairs/?sh=24e002951808>.
- [26] T. L. Foundation. SLSA: Supply-chain levels for software artifacts. <https://slsa.dev>, 2022. Accessed: 2022-04-30.
- [27] Git SCM. Signing your work. <https://git-scm.com/book/en/v2/Git-Tools-Signing-Your-Work>.
- [28] GitHub. Best practices for securing your build system. <https://docs.github.com/en/code-security/supply-chain-security/end-to-end-supply-chain/securing-builds>.
- [29] GitHub. Secure your supply chain, Retrieved July 30. <https://github.com/features/security/software-supply-chain>.
- [30] P. Goswami, S. Gupta, Z. Li, N. Meng, and D. Yao. Investigating the reproducibility of NPM packages. In *2020 IEEE International Conference on Software Maintenance and Evolution (ICSME)*, pages 677–681. IEEE.
- [31] S. T. A. Group. Software Supply Chain Best Practices. Technical report, Cloud Native Computing Foundation, May 2021.
- [32] S. T. A. Group. The Secure Software Factory: A reference architecture to securing the software supply chain. Technical report, Cloud Native Computing Foundation, June 2022.
- [33] R. He, H. He, Y. Zhang, and M. Zhou. Automating dependency updates in practice: An exploratory study on GitHub dependabot. 2022. <https://arxiv.org/abs/2206.07230>.
- [34] J. D. Herbsleb. Global software engineering: The future of socio-technical co-ordination. In *Future of Software Engineering (FOSE'07)*, pages 188–198. IEEE, 2007.
- [35] W. Humphrey. Characterizing the software process: a maturity framework. *IEEE Software*, 5(2):73–79, Mar. 1988. Conference Name: IEEE Software.
- [36] International Organization for Standardization and the International Electrotechnical Commission. ISO - ISO/IEC 27001 — information security management. <https://www.iso.org/isoiec-27001-information-security.html>.
- [37] International Organization for Standardization and the International Electrotechnical Commission. ISO/IEC 27002:2013 - code of practice for information security controls. <https://www.iso.org/cms/render/live/en/sites/isoorg/contents/data/standard/07/43/74399.html>.
- [38] International Organization for Standardization and the International Electrotechnical Commission. ISO/IEC 27002:2013- code of practice for information security controls. <https://www.iso.org/cms/render/live/en/sites/isoorg/contents/data/standard/05/45/54533.html>.
- [39] International Organization for Standardization and the International Electrotechnical Commission. ISO/IEC 27036-4:2016 - information security for supplier relationships. <https://www.iso.org/cms/render/live/en/sites/isoorg/contents/data/standard/05/96/59689.html>.
- [40] Joint Task Force Interagency Working Group. Security and Privacy Controls for Information Systems and Organizations. Technical report, National Institute of Standards and Technology, Sept. 2020. Edition: Revision 5.
- [41] Joint Task Force Transformation Initiative. Security and privacy controls for federal information systems and organizations. <https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-53r4.pdf>.
- [42] J. C. Knight and N. G. Leveson. An experimental evaluation of the assumption of independence in multiversion programming. *IEEE Transactions on Software Engineering*, SE-12(1):96–109.
- [43] P. Ladisa, H. Plate, M. Martinez, and O. Barais. Taxonomy of Attacks on Open-Source Software Supply Chains, 2022.
- [44] C. Lamb and S. Zacchiroli. Reproducible Builds: Increasing the Integrity of Software Supply Chains. *IEEE Software*, 39(2):62–70, Mar. 2022.
- [45] C. Liu, S. Chen, L. Fan, B. Chen, Y. Liu, and X. Peng. Demystifying the vulnerability propagation and its evolution via dependency trees in the NPM ecosystem. In *Proceedings of the 44th International Conference on Software Engineering*, pages 672–684. ACM, 2022.
- [46] L. Martin. Cyber Kill Chain®, June 2022.
- [47] W. M. McKeeman. Differential Testing for Software. 10(1):8, 1998.
- [48] Microsoft. 3 ways to mitigate risk when using private package feeds. Technical report, Microsoft, Mar. 2021.
- [49] Microsoft Security Response Center. Customer guidance on recent nation-state cyber attacks. <https://msrc-blog.microsoft.com/2020/12/13/customer-guidance-on-recent-nation-state-cyber-attacks/>.
- [50] National Telecommunications and Information Administration. NTIA Software Component Transparency. <https://www.ntia.doc.gov/SoftwareTransparency>.
- [51] Z. Newman, J. S. Meyers, and S. Torres-Arias. Sigstore: software signing for everybody. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*, 2021.
- [52] C. Nissen, J. E. Gronager, R. S. Metzger, and H. Rishikof. Deliver uncompromised: A strategy for supply chain security and resilience in response to the changing character of war. Technical report, MITRE CORP MCLEAN VA, 2018.
- [53] NPM. Proxy. <https://www.npmjs.com/package/proxy>.
- [54] NPM. Scope. <https://docs.npmjs.com/cli/v8/using-npm/scope>.
- [55] npm Docs. npm-audit. <https://docs.npmjs.com/cli/v8/commands/npm-audit/>.
- [56] N. I. of Standards and Technology. Minimum Security Requirements for Federal Information and Information Systems. Technical Report Federal Information Processing Standard (FIPS) 200, U.S. Department of Commerce, Mar. 2006.
- [57] M. Ohm, H. Plate, A. Sykosch, and M. Meier. Backstabber's knife collection: A review of open source software supply chain attacks. In *International Conference on Detection of Intrusions and Malware, and Vulnerability Assessment*, pages 23–43. Springer, 2020.
- [58] C. Paulsen, J. Boyens, N. Bartol, and K. Winkler. Criticality Analysis Process Model: Prioritizing Systems and Components. Technical Report NIST Internal or Interagency Report (NISTIR) 8179, National Institute of Standards and Technology, Apr. 2018.
- [59] G. A. A. Prana, A. Sharma, L. K. Shar, D. Foo, A. E. Santosa, A. Sharma, and D. Lo. Out of sight, out of mind? how vulnerable dependencies affect open-source projects. *Empirical Software Engineering*, 26(4):1–34, 2021.
- [60] E. Raymond. The cathedral and the bazaar. *Knowledge, Technology & Policy*, 12(3):23–49, 1999.

- [61] T. J. Ronda, P. A. Roberge, D. Barinov, M. Varley, D. A. Stark, G. H. Wolfond, A. Likic, and M. J. Page. Systems and methods for distributed identity verification, Mar. 19 2019. US Patent 10,237,259.
- [62] R. Ross, M. McEvilly, and J. C. Oren. Systems security engineering: considerations for a multidisciplinary approach in the engineering of trustworthy secure systems, volume 1.
- [63] A. Sejfia and M. Schäfer. Practical automated detection of malicious npm packages. In *2022 International Conference on Software Engineering (ICSE)*. IEEE, 2022.
- [64] D. Šmite, C. Wohlin, T. Gorschek, and R. Feldt. Empirical evidence in global software engineering: a systematic review. *Empirical software engineering*, 15(1):91–118, 2010.
- [65] Solarwinds. Setting the New Standard in Secure Software Development The SolarWinds Next-Generation Build System. Technical report, solarwinds, Dec. 2021.
- [66] Sonatype. State of the software supply chain, 2021. <https://www.sonatype.com/resources/state-of-the-software-supply-chain-2021>.
- [67] H. Stadler and C. Kilger, editors. *Supply Chain Management and Advanced Planning*. Springer Berlin Heidelberg, Berlin, Heidelberg, 4th edition, 2008.
- [68] Symantec Corporation. Internet threat security report, 2018. <https://www.symantec.com/content/dam/symantec/docs/reports/istr-23-2018-en.pdf>.
- [69] Symantec Corporation. Internet threat security report, 2019. <https://www.symantec.com/content/dam/symantec/docs/reports/istr-24-2019-en.pdf>.
- [70] N. Telecommunications and I. Administration. The Minimum Elements For a Software Bill of Materials (SBOM), July 2021. <https://www.ntia.doc.gov/report/2021/minimum-elements-software-bill-materials-sbom>.
- [71] K. Thomas, J. Pullman, K. Yeo, A. Raghunathan, P. G. Kelley, L. Invernizzi, B. Benko, T. Pietraszek, S. Patel, D. Boneh, and E. Bursztein. Protecting accounts from credential stuffing with password breach alerting. page 18.
- [72] S. Torres-Arias, H. Afzali, T. K. Kuppusamy, R. Curtmola, and J. Capps. in-toto: Providing farm-to-table guarantees for bits and bytes. In *28th USENIX Security Symposium (USENIX Security 19)*, pages 1393–1410, Santa Clara, CA, Aug. 2019. USENIX Association.
- [73] S. Torres-Arias, H. Afzali, T. K. Kuppusamy, R. Curtmola, and J. Capps. in-toto: Providing farm-to-table guarantees for bits and bytes. *Proc. of the 28th USENIX Security Symposium*, Aug. 2019.
- [74] N. Vasilakis, B. Karel, N. Roessler, N. Dautenhahn, A. DeHon, and J. M. Smith. BreakApp: Automated, Flexible Application Compartmentalization. In *Proceedings 2018 Network and Distributed System Security Symposium*, San Diego, CA, 2018. Internet Society.
- [75] D.-L. Vu, F. Massacci, I. Pashchenko, H. Plate, and A. Sabetta. LastPyMile: identifying the discrepancy between sources and packages. In *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, pages 780–792, Athens Greece, Aug. 2021. ACM.
- [76] T. Winters, T. Manshreck, and H. Wright. *Software engineering at google: Lessons learned from programming over time*. O'Reilly Media, 2020.
- [77] Q. Wu and K. Lu. On the feasibility of stealthily introducing vulnerabilities in open-source software via hypocrite commits. In *Proc. Oakland*, 2021.
- [78] E. Wyss, L. De Carli, and D. Davidson. What the fork?: finding hidden code clones in npm. In *Proceedings of the 44th International Conference on Software Engineering*, pages 2415–2426. ACM, 2022.
- [79] N. Zahan, T. Zimmermann, P. Godefroid, B. Murphy, C. Maddila, and L. Williams. What are Weak Links in the npm Supply Chain?, 2022.
- [80] A. Zerouali, T. Mens, A. Decan, and C. De Roover. On the impact of security vulnerabilities in the npm and rubygems dependency networks. *Empirical Software Engineering*, 27(5):1–45, 2022.
- [81] M. Zimmermann, C.-A. Staicu, and M. Pradel. Small World with High Risks: A Study of Security Threats in the npm Ecosystem. In *USENIX Security Symposium*, 2019.